
GARITA TECNOLOGICA

Sistema de Control de Acceso Residencial

MANUAL TECNICO

1. Descripción General del Sistema

Garita tecnológica es una API REST desarrollada con FastAPI (Python) que gestiona el control de acceso de visitantes a un residencial. Integra autenticación de usuarios con roles diferenciados (IAM), almacenamiento persistente en PostgreSQL mediante SQLAlchemy ORM, y un servicio de notificaciones por correo electrónico con soporte para códigos QR.

1.1 Stack Tecnológico

Componente	Tecnologia	Version / Notas
Framework principal	FastAPI	Python 3.10+
ORM / Base de datos	SQLAlchemy + PostgreSQL	pool_pre_ping activado
Validacion de datos	Pydantic v2	schemas.py
Servicio de correo	smtplib (SMTP/TLS)	Gmail SMTP, puerto 587
Generacion de QR	qrcode	Imagen PNG incrustada en email
Identificadores unicos	uuid (stdlib)	Tokens de invitacion y codigos de residente
Migraciones	Creacion automatica de tablas	Base.metadata.create_all

1.2 Estructura del Proyecto

```
garita/  
  database.py      # Motor SQLAlchemy, sesion y dependencias  
  models.py       # Modelos ORM (tablas): Usuario, Residente, Visitante, Acceso  
  schemas.py      # Esquemas Pydantic para validacion de entrada/salida  
  crud.py         # Logica de negocio y operaciones de base de datos  
  email_service.py # Servicio de notificaciones SMTP con QR  
  main.py         # Definicion de rutas FastAPI (endpoints)
```

2. Modelo de Base de Datos

2.1 Configuración de Conexión (database.py)

La conexión a PostgreSQL se establece mediante SQLAlchemy con las siguientes configuraciones de seguridad y rendimiento:

```
SQLALCHEMY_DATABASE_URL = 'postgresql://postgres:1234@localhost:5432/garita_db'

engine = create_engine(
    SQLALCHEMY_DATABASE_URL,
    pool_pre_ping=True, # Evita conexiones muertas
    pool_size=5,         # Conexiones simultaneas maximas
    max_overflow=10      # Conexiones extra permitidas
)
```

PRODUCCION: Mover credenciales a variables de entorno (.env) antes de despliegue. Usar python-dotenv o equivalente. Nunca exponer contraseñas en el repositorio.

2.2 Diagrama de Entidades

Tabla	Clave Primaria	Claves Foraneas	Descripcion
usuarios	id (PK)	-	Cuentas de acceso al sistema (admin, agente, vecino)
residentes	id (PK)	usuario_id -> usuarios.id	Propietarios o residentes del conjunto
visitantes	id (PK)	-	Personas que ingresan al residencial
accesos	id (PK)	visitante_id, residente_id, agente_id -> usuarios.id	Registro de cada ingreso/egreso

2.3 Modelos ORM Detallados (models.py)

Tabla: usuarios

Campo	Tipo	Restricciones	Descripcion
id	Integer	PK, index	Identificador auto-incremental
username	String	UNIQUE, NOT NULL	Correo electronico del usuario
password_hash	String	NOT NULL	Contraseña (texto plano en v0.5 - pendiente hashing)
rol	String	-	Valores: admin agente vecino

Tabla: residentes

Campo	Tipo	Restricciones	Descripcion
id	Integer	PK, index	Identificador auto-incremental
nombre_propietario	String	NOT NULL	Nombre completo del propietario
numero_casa	String	UNIQUE, NOT NULL	Identificador de la unidad habitacional
codigo_unico	String	UNIQUE, NOT NULL	Codigo QR personal (ej. REG-A3B2)
correo_notificacion	String	NOT NULL	Email para recibir alertas de visita
usuario_id	Integer	FK usuarios.id, UNIQUE, nullable	Vinculacion IAM

Tabla: accesos

Campo	Tipo	Restricciones	Descripcion
id	Integer	PK, index	Identificador auto-incremental
visitante_id	Integer	FK visitantes.id	Visitante asociado
residente_id	Integer	FK residentes.id	Residente que recibe la visita

Campo	Tipo	Restricciones	Descripcion
agente_id	Integer	FK usuarios.id, nullable	Agente que proceso el acceso
fecha_entrada	DateTime	server_default=now()	Timestamp de ingreso
fecha_salida	DateTime	nullable	Timestamp de egreso (null si aun dentro)
tipo_acceso	String	-	MANUAL o QR

3. Endpoints de la API

3.1 Autenticacion y Usuarios

Metodo	Ruta	Descripcion	Roles Permitidos
POST	/usuarios/	Crear nuevo usuario en el sistema	admin
GET	/usuarios/	Listar todos los usuarios registrados	admin
POST	/login	Autenticar usuario y obtener datos de sesion	Todos

Respuesta de /login (exitoso):

```
{
  "id": 1,
  "username": "vecino@correo.com",
  "rol": "vecino",
  "residente_id": 3
}
```

SEGURIDAD PENDIENTE: El endpoint /login compara la contraseña en texto plano. Se debe implementar hashing con passlib/bcrypt antes de produccion. Tambien se recomienda implementar JWT para manejo de sesiones.

3.2 Gestion de Residentes

Metodo	Ruta	Descripcion	Efecto Secundario
POST	/residentes/	Registrar nuevo residente	Crea usuario IAM + envia email bienvenida
GET	/residentes/	Listar todos los residentes	-
GET	/residentes/qr/{id}	Generar imagen QR del residente	Retorna imagen PNG via streaming

Payload de registro de residente:

```
{
  "nombre_propietario": "Juan Perez",
  "numero_casa": "Casa-05",
  "correo_notificacion": "juan@correo.com"
}
```

3.3 Control de Accesos (Guardia)

Metodo	Ruta	Descripcion	Efecto Secundario
GET	/accesos/	Obtener historial completo de accesos	-
POST	/accesos/	Generar invitacion QR desde vecino	Envia QR por correo al invitado
POST	/accesos/entrada	Registrar ingreso manual en garita	Notifica al residente por email
PUT	/accesos/salida/{id}	Registrar salida de visitante	Actualiza fecha_salida en DB

Payload de registro de entrada manual:

```
{
  "nombre_visitante": "Maria Lopez",
  "placa_vehiculo": "ABC-123",
  "residente_id": 3,
  "tipo_acceso": "MANUAL"
}
```

3.4 Dashboard

Metodo	Ruta	Descripcion
GET	/dashboard/stats	Retorna total de residentes, visitas activas y alertas de seguridad

Respuesta de /dashboard/stats:

```
{
  "residentes_totales": 12,
```

```
"visitas_activas": 3,  
"alertas_seguridad": 0  
}
```

4. Logica de Negocio (crud.py)

4.1 Registro de Residentes

El proceso de registro crea automáticamente el usuario IAM asociado con el correo como username y el número de casa como contraseña temporal. El Código único se auto-genera con el formato REG-XXXX si no se proporciona.

```
def crear_residente(db, residente, usuario_id=None):
    cod_final = residente.codigo_unico or f'REG-{uuid.uuid4().hex[:4].upper()}'
    db_residente = models.Residente(
        nombre_propietario=residente.nombre_propietario,
        numero_casa=residente.numero_casa,
        codigo_unico=cod_final,
        correo_notificacion=residente.correo_notificacion,
        usuario_id=usuario_id
    )
    db.add(db_residente)
    db.commit()
    db.refresh(db_residente)
    return db_residente
```

4.2 Registro de Entrada de Visitante

El proceso es transaccional: primero crea el registro en la tabla visitantes, luego crea el acceso vinculado. Los datos del visitante se inyectan en el objeto de acceso para que el schema Pydantic pueda serializarlos correctamente.

4.3 Consulta de Visitas Activas

Filtra los accesos con fecha_salida == None. Gracias a las relaciones SQLAlchemy, el visitante se carga automáticamente (lazy loading) y sus datos se mapean a atributos dinámicos del objeto acceso para la respuesta JSON.

4.4 Búsqueda por Código

La función obtener_residente_por_codigo acepta tanto el numero de casa como el Código único, usando un OR lógico en la consulta SQL:

```
db.query(models.Residente).filter(
    or_(
        models.Residente.numero_casa == codigo,
        models.Residente.codigo_unico == codigo
    )
)
```

```
).first()
```

5. Servicio de Notificaciones por Correo (email_service.py)

5.1 configuración SMTP

Parametro	Valor
Servidor	smtp.gmail.com
Puerto	587 (STARTTLS)
Autenticación	App Password de Google (16 caracteres)
Formato del remitente	formataddr() para header correcto

SEGURIDAD: La contraseña de aplicación SMTP esta hardcoded en el Código.
Mover a variable de entorno SMTP_PASSWORD antes de subir a un repositorio.
Utilizar python-dotenv: SENDER_PASSWORD = os.getenv('SMTP_PASSWORD')

5.2 Funciones de Notificación

Funcion	Cuando se llama	Contenido del correo
enviar_correo_bienvenida()	Al registrar un nuevo residente	HTML con credenciales de acceso al portal
enviar_notificacion_visita()	Al registrar entrada manual en garita	Texto plano con nombre del visitante y destino
enviar_correo_con_qr()	Al generar invitacion desde portal del vecino	HTML con codigo QR incrustado (Content-ID)

5.3 Generación del Código QR

El QR se genera en memoria usando la librería qrcode, se guarda en un buffer BytesIO en formato PNG y se adjunta al correo como imagen inline mediante Content-ID para que aparezca embebido en el HTML.

```
qr = qrcode.QRCode(version=1, box_size=10, border=5)
qr.add_data(token)
qr.make(fit=True)
img = qr.make_image(fill_color='black', back_color='white')
```

```
buffer = BytesIO()
img.save(buffer, format='PNG')

image_attachment = MIMEImage(buffer.getvalue())
image_attachment.add_header('Content-ID', '<qr_code>')
image_attachment.add_header('Content-Disposition', 'inline',
filename='codigo_acceso.png')
```

6. Seguridad - Estado Actual y Recomendaciones

6.1 Vulnerabilidades Identificadas en v0.5

Vulnerabilidad	Archivo	Riesgo	Recomendacion
contraseñas en texto plano	crud.py, main.py	ALTO	Implementar passlib con bcrypt
Credenciales SMTP hardcoded	email_service.py	ALTO	Usar variables de entorno (.env)
Credenciales DB hardcoded	database.py	ALTO	Usar variables de entorno
Sin autenticación JWT	main.py	ALTO	Implementar python-jose + OAuth2
CORS allow_origins=[*]	main.py	MEDIO	Restringir a dominios autorizados
Sin rate limiting	main.py	MEDIO	Usar slowapi o middleware personalizado
Sin validación de roles por endpoint	main.py	ALTO	Implementar dependencias de autorización
Token QR sin expiración	main.py	MEDIO	Agregar campo expires_at al modelo

6.2 Implementación de Hashing de contraseñas

Para implementar hashing seguro con bcrypt en una version futura:

```
# pip install passlib[bcrypt]
from passlib.context import CryptContext

pwd_context = CryptContext(schemes=['bcrypt'], deprecated='auto')

def hash_password(password: str) -> str:
    return pwd_context.hash(password)

def verify_password(plain: str, hashed: str) -> bool:
    return pwd_context.verify(plain, hashed)
```

6.3 Implementacion de JWT

```
# pip install python-jose[cryptography]
from jose import JWTError, jwt
from datetime import datetime, timedelta

SECRET_KEY = os.getenv('SECRET_KEY')
ALGORITHM = 'HS256'
ACCESS_TOKEN_EXPIRE_MINUTES = 60

def create_access_token(data: dict):
    expire = datetime.utcnow() + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    data.update({'exp': expire})
    return jwt.encode(data, SECRET_KEY, algorithm=ALGORITHM)
```

7. Guia de Instalación y Despliegue

7.1 Requisitos del Sistema

Componente	Version Minima	Notas
Python	3.10+	Recomendado 3.11+
PostgreSQL	13+	Crear BD: garita_db
pip	23+	Administrador de paquetes

7.2 Instalación de Dependencias

```
# Crear entorno virtual
python -m venv venv
source venv/bin/activate # Linux/Mac
.\venv\Scripts\activate # Windows

# Instalar dependencias
pip install fastapi uvicorn sqlalchemy psycpg2-binary
pip install pydantic[email] qrcode[pil] pillow
pip install python-dotenv passlib[bcrypt]
```

7.3 Variables de Entorno (.env)

```
# .env
DATABASE_URL=postgresql://postgres:PASSWORD@localhost:5432/garita_db
SMTP_SERVER=smtp.gmail.com
SMTP_PORT=587
SENDER_EMAIL=tu_correo@gmail.com
SENDER_PASSWORD=xxxx_xxxx_xxxx_xxxx
SECRET_KEY=una_clave_segura_aleatoria
```

7.4 Iniciar el Servidor

```
# Desde el directorio raiz del proyecto
uvicorn garita.main:app --reload --host 0.0.0.0 --port 8000

# Documentacion interactiva disponible en:
# http://localhost:8000/docs (Swagger UI)
# http://localhost:8000/redoc (ReDoc)
```

7.5 Creación de la Base de Datos

Las tablas se crean automáticamente al iniciar el servidor gracias a:

```
# En main.py
models.Base.metadata.create_all(bind=engine)
```

Para despliegues en producción se recomienda usar Alembic para migraciones controladas.

8. Flujos de Proceso del Sistema

8.1 Flujo de Registro de Residente

Paso	Accion	Modulo
1	Admin envia POST /residentes/ con datos del residente	main.py
2	Se verifica que el correo no este registrado como usuario	main.py
3	Se crea usuario IAM con rol 'vecino' (username=correo, password=numero_casa)	crud.py
4	Se crea el residente con codigo_unico auto-generado (REG-XXXX)	crud.py
5	Se envia correo de bienvenida con credenciales al residente	email_service.py
6	Se retorna el objeto residente creado (schema Pydantic)	main.py

8.2 Flujo de Acceso Manual en Garita

Paso	Accion	Modulo
1	Agente envia POST /accesos/entrada con datos del visitante y residente_id	main.py
2	Se crea registro en tabla visitantes	crud.py
3	Se crea registro en tabla accesos con fecha_entrada automatica (server_default)	crud.py
4	Se consulta el residente para obtener su correo de notificacion	main.py
5	Se envia notificacion automatica al residente por correo	email_service.py
6	Se retorna el acceso con datos del visitante inyectados	main.py

8.3 Flujo de Invitacion QR desde Portal de Vecino

Paso	Accion	Modulo
1	Residente envia POST /accesos/ con datos del invitado y correo de destino	main.py
2	Se crea visitante en DB	main.py
3	Se genera token unico: INV-XXXXXX-CasaXX	main.py
4	Se crea acceso con tipo_acceso='QR' y fecha_entrada=None (pendiente)	main.py
5	Se genera imagen QR con el token y se envia por correo al invitado	email_service.py
6	Invitado presenta QR en garita; agente valida token y autoriza ingreso	Proceso manual

9. Glosario Técnico

Termino	Definicion
FastAPI	Framework web Python moderno para construcción de APIs REST con validación automática via Pydantic.
SQLAlchemy	ORM Python que abstrae operaciones SQL y permite trabajar con modelos Python en lugar de SQL directo.
Pydantic	Librería de validación de datos usada por FastAPI para validar y serializar schemas de entrada/salida.
ORM	Object-Relational Mapping: técnica de mapeo entre objetos Python y tablas de base de datos relacional.
SMTP/STARTTLS	Protocolo de correo electrónico con cifrado TLS activado dinámicamente en el puerto 587.
Content-ID (CID)	Mecanismo MIME para referenciar imágenes adjuntas directamente en el HTML de un correo electrónico.
pool_pre_ping	parámetro SQLAlchemy que verifica conexiones antes de usarlas, evitando errores por conexiones expiradas.
Lazy Loading	Estrategia de carga de relaciones en SQLAlchemy: los datos relacionados se consultan al ser accedidos.
UUID	Identificador Unico Universal de 128 bits generado aleatoriamente, usado para tokens de invitacion.
JWT	JSON Web Token: estandar para transmitir informacion de autentificacion de forma segura y compacta.
bcrypt	Algoritmo de hashing de contraseñas con factor de costo ajustable, recomendado para produccion.
Alembic	Herramienta de migraciones de base de datos para SQLAlchemy, recomendada para entornos de produccion.